



Qwen2.5-1M: 支持100万Token上下文的开源Qwen模型

2025年1月27日 · 3 分钟 · 501 字 · Qwen Team | 语言: [English](#)

[TECH REPORT](#)[HUGGINGFACE](#)[MODELSCOPE](#)[QWEN CHAT](#)[HUGGINGFACE DEMO](#)[MODELSCOPE DEMO](#)[DISCORD](#)

简介

两个月前，我们升级了 [Qwen2.5-Turbo](#)，使其支持最多一百万个Tokens的上下文长度。今天，我们正式推出开源的 Qwen2.5-1M 模型及其对应的推理框架支持。以下是本次发布的亮点：

- 开源模型：**我们发布了两个新的开源模型，分别是 **Qwen2.5-7B-Instruct-1M** 和 **Qwen2.5-14B-Instruct-1M**，这是我们首次将开源 Qwen 模型的上下文扩展到 1M 长度。
- 推理框架：**为了帮助开发者更高效地部署 Qwen2.5-1M 系列模型，我们完全开源了基于 [vLLM](#) 的推理框架，并集成了稀疏注意力方法。该框架在处理 1M 长度输入时的速度能够提升 **3倍到7倍**。
- 技术报告：**我们还分享了 Qwen2.5-1M 系列背后的技术细节，包括训练和推理框架的设计思路以及消融实验的结果。

现在，你可以访问我们在 [Huggingface](#) 和 [Modelscope](#) 上的在线演示来体验 Qwen2.5-1M 模型。

另外，我们最近也推出了 [Qwen Chat](#)，一个基于 Qwen 系列的 AI 助手。你可以与他对话、编程、生成图像与视频，使用搜索以及调用工具等功能。你也可以在 Qwen Chat 中与使用上下文长度同样为 1M 的 Qwen2.5-Turbo 模型进行长序列处理。

模型性能

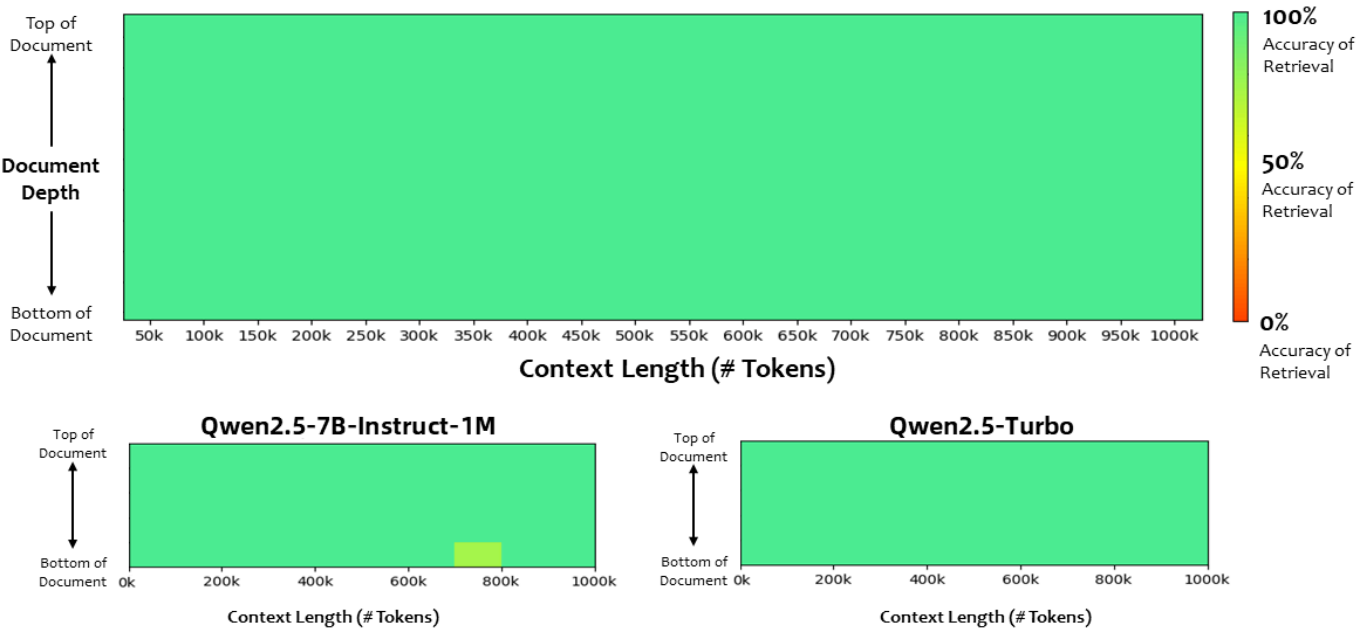
首先，让我们来看看 Qwen2.5-1M 系列模型在长上下文任务和短文本任务中的性能表现。

长上下文任务



Testing Qwen2.5-14B-Instruct-1M via “Passkey Retrieval”

Retrieve Hidden Number from Irrelevant Sentences across Context Lengths and Document Depth



对于更复杂的长上下文理解任务，我们选择了RULER、LV-Eval和LongbenchChat，这些测试集也在此博客中进行了介绍。

Model		Claimed Length	RULER						
			Avg.	4K	8K	16K	32K	64K	128K
GLM4-9b-Chat-1M		1M	89.9	94.7	92.8	92.1	89.9	86.7	83.1
Llama-3-8B-Instruct-Gradient-1048k		1M	88.3	95.5	93.8	91.6	87.4	84.7	77.0
Llama-3.1-70B-Instruct		128K	89.6	96.5	95.8	95.4	94.8	88.4	66.6
GPT-4o-mini		128K	87.3	95.0	92.9	92.7	90.2	87.6	65.8
GPT-4		128K	91.6	96.6	96.3	95.2	93.2	87.0	81.2
Qwen2.5-32B-Instruct	RoPE	32K	88.0	96.9	97.1	95.5	95.5	85.3	57.7
	DCA+YaRN	128K	92.9					90.3	82.0
Qwen2.5-72B-Instruct	RoPE	32K	90.8	97.7	97.2	97.7	96.5	88.5	67.0
	DCA+YaRN	128K	95.1					93.0	88.4
Qwen2.5-7B-Instruct	RoPE	32K	80.1	96.7	95.1	93.7	89.4	74.5	31.4
	DCA+YaRN	128K	85.4					82.3	55.1
Qwen2.5-7B-Instruct-1M		1M	91.8	96.8	95.3	93.0	91.1	90.4	84.4
Qwen2.5-14B-Instruct	RoPE	32K	86.5	97.7	96.8	95.9	93.4	82.3	53.0
	DCA+YaRN	128K	91.4					86.7	78.1
Qwen2.5-14B-Instruct-1M		1M	95.7	97.5	97.1	94.6	94.9	94.9	92.2
Qwen2.5-Turbo		1M	93.1	97.5	95.7	95.5	94.8	90.8	84.5



GLM4-9B-Chat-1M		1M	46.4	43.2	42.9	40.4	37.0	7.82
Llama-3-8B-Instruct-Gradient-1048k		1M	31.7	31.8	28.8	26.3	21.1	6.20
Llama-3.1-70B-Instruct		128K	48.6	47.4	42.9	26.2	N/A	6.80
GPT-4o-mini		128K	52.9	48.1	46.0	40.7	N/A	8.48
Qwen2.5-32B-Instruct	RoPE	32K			40.1	20.5	0.7	-
	DCA+YaRN	128K	56.0	53.6	48.8	45.3	41.0	8.70
Qwen2.5-72B-Instruct	RoPE	32K			47.4	27.0	2.4	-
	DCA+YaRN	128K	60.4	57.5	53.9	50.9	45.2	8.72
Qwen2.5-7B-Instruct	RoPE	32K			33.1	13.6	0.5	-
	DCA+YaRN	128K	55.9	49.7	48.0	41.1	36.9	7.42
Qwen2.5-7B-Instruct-1M	RoPE / DCA+YaRN	1M	52.5	49.4	48.6	48.3	42.7	8.08
Qwen2.5-14B-Instruct	RoPE	32K			37.0	18.4	0.8	-
	DCA+YaRN	128K	53.0	50.8	46.8	43.6	39.4	8.04
Qwen2.5-14B-Instruct-1M	RoPE / DCA+YaRN	1M	54.5	53.5	50.1	47.6	43.3	8.76
Qwen2.5-Turbo	RoPE / DCA+YaRN	1M	53.4	50.0	45.4	43.9	38.0	8.34

从这些结果中，我们可以得出以下几点关键结论：

- 1. **显著超越128K版本**：Qwen2.5-1M 系列模型在大多数长上下文任务中显著优于之前的128K版本，特别是在处理超过64K长度的任务时表现出色。
- 2. **性能优势明显**：Qwen2.5-14B-Instruct-1M 模型不仅击败了 Qwen2.5-Turbo，还在多个数据集上稳定超越 GPT-4o-mini，因此可以作为现有长上下文模型的优秀开源替代。

短序列任务

除了长序列任务的性能外，我们同样关注这些模型在短序列上的表现。我们在广泛使用的学术基准测试中比较了 Qwen2.5-1M 系列模型及之前的128K版本，并加入了 GPT-4o-mini 进行对比。

Datasets	GPT4o-mini	Qwen2.5-7B	Qwen2.5-7B-1M	Qwen2.5-14B	Qwen2.5-14B-1M	Qwen2.5-Turbo
General Tasks						
MMLU-Pro	63.1	56.3	54.3	63.7	63.3	64.5
MMLU-redux	81.5	75.4	74.8	80.0	80.7	81.7
LiveBench 0831	31.1	35.9	35.2	44.4	44.6	42.3
Mathematics & Science Tasks						
GPQA	40.2	36.4	41.4	45.5	39.9	42.3
MATH	70.2	75.5	72.9	80.0	79.5	81.1
GSM8K	93.2	91.6	91.7	94.8	94.8	93.8
Coding Tasks						
HumanEval	88.4	84.8	86.0	83.5	88.4	86.6
MBPP	85.7	79.2	75.9	82.0	80.2	82.8
MultiPL-E	75.0	70.4	72.4	72.8	77.1	73.7
LiveCodeBench	40.7	28.7	28.0	42.6	38.6	37.8
Alignment Tasks						
IFEval	80.4	71.2	73.0	81.0	84.3	76.3
Arena-Hard	74.9	52.0	48.1	68.3	70.2	67.1
MTbench	-	8.75	8.30	8.88	8.89	8.81

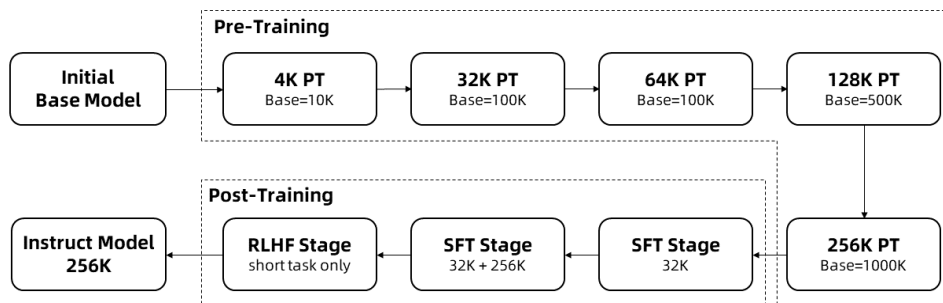
我们可以发现：

- Qwen2.5-7B-Instruct-1M 和 Qwen2.5-14B-Instruct-1M 在短文本任务上的表现与其128K版本相当，确保了基本能力没有因为增加了长序列处理能力而受到影响。
- 与 GPT-4o-mini 相比，Qwen2.5-14B-Instruct-1M 和 Qwen2.5-Turbo 在短文本任务上实现了相近的性能，同时上下文长度是 GPT-4o-mini 的八倍。



在这里，我们将简要介绍构建 Qwen2.5-1M 背后的关键技术。更多内容可参阅我们的[技术报告](#)。

长上下文训练



长序列的训练需要大量的计算资源，因此我们采用了逐步扩展长度的方法，在多个阶段将 Qwen2.5-1M 的上下文长度从 4K 扩展到 256K：

- 我们从预训练的Qwen2.5的一个中间检查点开始，此时上下文长度为4K。
- 在预训练阶段，我们逐步将上下文长度从 4K 增加到 256K，同时使用[Adjusted Base Frequency](#)的方案，将 RoPE 基础频率从 10,000 提高到 10,000,000。
- 在监督微调阶段，我们分两个阶段进行以保持短序列上的性能：
 - 第一阶段：仅在短指令（最多 32K 长度）上进行微调，这里我们使用与 Qwen2.5 的 128K 版本相同的数据和步骤数，以获得类似的短任务性能。
 - 第二阶段：混合短指令（最多 32K）和长指令（最多 256K）进行训练，以实现在增强长任务的性能的同时，保持短任务上的准确率。
- 在强化学习阶段，我们在短文本（最多 8K 长度）上训练模型。我们发现，即使在短文本上进行训练，也能很好地将人类偏好对齐性能泛化到长上下文任务中。

通过以上训练，我们最终获得了 256K 上下文长度的指令微调模型。

长度外推

在上述训练过程中，模型的上下文长度仅为 256K 个 Tokens。为了将其扩展到 1M，我们采用了长度外推的技术。

当前，基于旋转位置编码的大型语言模型会在长上下文任务中产生性能下降，这主要是由于在计算注意力权重时，Query 和 Key 之间的相对位置距离过大，在训练过程中未曾见过。为了解决这一问题，我们引入了[Dual Chunk Attention](#) (DCA)，该方法通过将过大的相对位置，重新映射为较小的值，从而解决了这一难题。



	2	1	0																
	3	2	1	0															
	4	3	2	1	0														
	5	4	3	2	1	0													
	6	5	4	3	2	1	0												
	7	6	5	4	3	2	1	0											
	8	7	6	5	4	3	2	1	0										
	9	8	7	6	5	4	3	2	1	0									
	10	9	8	7	6	5	4	3	2	1	0								
	11	10	9	8	7	6	5	4	3	2	1	0							
	12	11	10	9	8	7	6	5	4	3	2	1	0						
	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				

Query

(a) Unadjusted Relative Positional Matrix

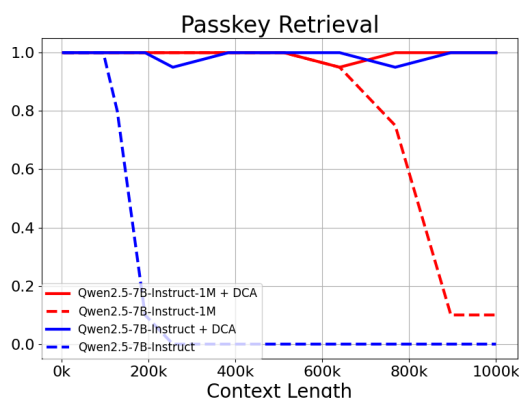
	2	1	0																
	3	2	1	0															
	4	3	2	1	0														
	5	4	3	2	1	0													
	6	5	4	3	2	1	0												
	7	6	5	4	3	2	1	0											
	8	7	6	5	4	3	2	1	0										
	9	8	7	6	5	4	3	2	1	0									
	10	9	8	7	6	5	4	3	2	1	0								
	11	10	9	8	7	6	5	4	3	2	1	0							
	12	11	10	9	8	7	6	5	4	3	2	1	0						
	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				

Query

(b) Relative Positional Matrix in DCA

我们对 Qwen2.5-1M 模型及之前 128K 的版本进行了评估，分别测试了使用和不使用长度外推方法的情况。

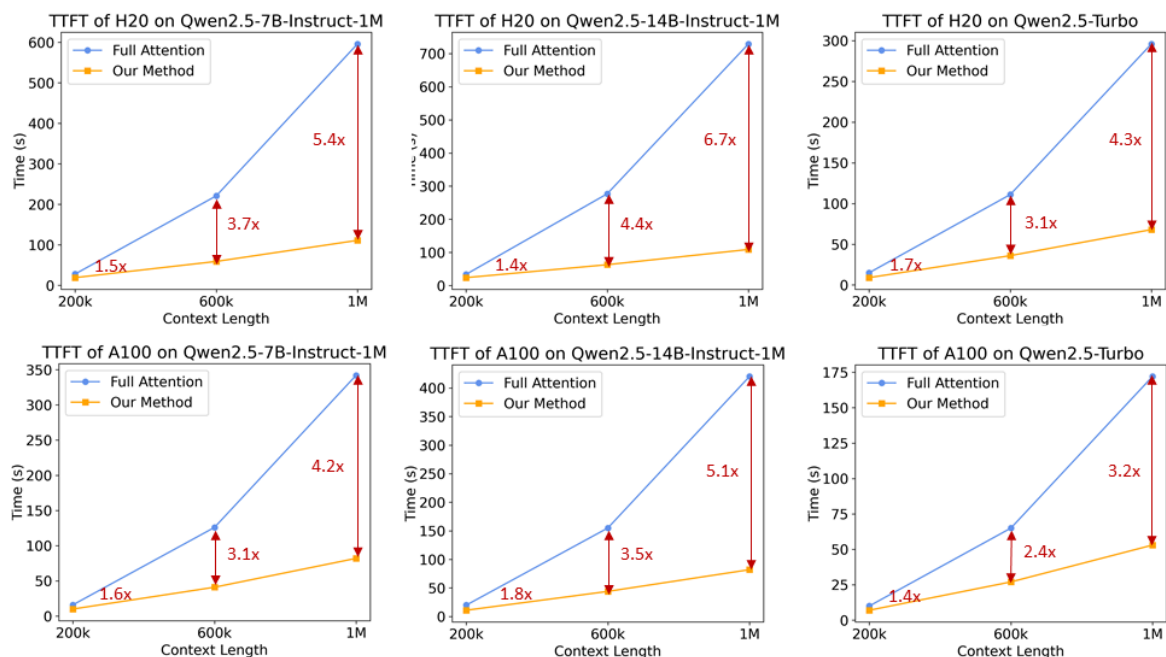
结果表明：即使是仅在 32K 长度上训练的 Qwen2.5-7B-Instruct，在处理 1M 上下文的 Passkey Retrieval 任务中也能达到近乎完美的准确率。这充分展示了 DCA 在无需额外训练的情况下，也可显著扩展支持的上下文长度的强大能力。



稀疏注意力机制

对于长上下文的语言模型，推理速度对用户体验至关重要。为了加速预填充阶段，我们引入了基于 **Minference** 的稀疏注意力优化。在此基础上，我们还提出了一系列改进：

- 分块预填充：** 如果直接使用模型处理长度100万的序列，其中 MLP 层的激活权重会产生巨大的显存开销。以 Qwen2.5-7B 为例，这部分开销高达 71GB。通过将分块预填充（Chunked Prefill）与稀疏注意力适配，可以将输入序列以 32768 长度分块，逐块进行预填充，MLP 层激活权重的显存使用量可减少 96.7%，因而显著降低了设备的显存需求。
- 集成长度外推方案：** 我们在稀疏注意力机制中进一步集成了基于 DCA 的长度外推方案，这使我们的推理框架能够同时享受更高的推理效率和长序列任务的准确性。
- 稀疏性优化：** 原始的 Minference 方法需要进行离线搜索以确定每个注意力头的最佳稀疏化配置。由于全注意力权重对内存的要求太大，这种搜索通常在短序列上进行，不一定能在更长序列下起到很好的效果。我们提出了一种能够在100万长度的序列上优化稀疏化配置的方法，从而显著减少了稀疏注意力带来的精度损失。
- 其他优化：** 我们还引入了其他优化措施，如优化算子效率和动态分块流水线并行，以充分发挥整个框架的潜力。



本地部署 Qwen2.5-1M 模型

接下来，我们将逐步介绍如何将 Qwen2.5-1M 模型部署到本地设备。

1. 系统准备

为了获得最佳性能，我们建议使用支持优化内核的 Ampere 或 Hopper 架构的 GPU。

请确保满足以下要求：

- **CUDA 版本**：12.1 或 12.3
- **Python 版本**：>=3.9 且 <=3.12

显存要求，对于处理 1M 长度的序列：

- **Qwen2.5-7B-Instruct-1M**：至少需要 120GB 显存（多 GPU 总和）。
- **Qwen2.5-14B-Instruct-1M**：至少需要 320GB 显存（多 GPU 总和）。

如果 GPU 显存不满足以上要求，你仍然可以使用 Qwen2.5-1M 进行较短任务的处理。

2. 安装依赖项

暂时，你需要从我们的自定义分支克隆 vLLM 仓库，并手动安装。我们正在努力将我们的分支提交到 vLLM 项目中。

```
git clone -b dev/dual-chunk-attn git@github.com:QwenLM/vllm.git
cd vllm
pip install -e . -v
```

3. 启动 OpenAI 兼容的 API 服务



```
vllm serve Qwen/Qwen2.5-7B-Instruct-1M \
--tensor-parallel-size 4 \
--max-model-len 1010000 \
--enable-chunked-prefill --max-num-batched-tokens 131072 \
--enforce-eager \
--max-num-seqs 1

# --quantization fp8 # you can use fp8 quantization for model weights to reduce memory usage
```

参数说明：

- **--tensor-parallel-size**
 - 设置为您使用的 GPU 数量。7B 模型最多支持 4 个 GPU，14B 模型最多支持 8 个 GPU。
- **--max-model-len**
 - 定义最大输入序列长度。如果遇到内存不足问题，请减少此值。
- **--max-num-batched-tokens**
 - 设置 Chunked Prefill 的块大小。较小的值可以减少激活内存使用，但可能会减慢推理速度。
 - 推荐值为 131072，以获得最佳性能。
- **--max-num-seqs**
 - 限制并发处理的序列数量。

如果遇到问题，请参考 [Troubleshooting](#) 相关内容。

4. 与模型交互

你可以使用以下方法与部署的模型进行交互：

选项 1. 使用 Curl

```
curl http://localhost:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "Qwen/Qwen2.5-7B-Instruct-1M",
  "messages": [
    {"role": "user", "content": "告诉我一些关于大型语言模型的事情。"}
  ],
  "temperature": 0.7,
  "top_p": 0.8,
  "repetition_penalty": 1.05,
  "max_tokens": 512
}'
```

选项 2. 使用 Python

```
from openai import OpenAI

openai_api_key = "EMPTY"
openai_api_base = "http://localhost:8000/v1"

client = OpenAI(
    api_key=openai_api_key,
    base_url=openai_api_base,
)
```



```
"The pass key is 28884. Remember it. 28884 is the pass key.\n"
"The grass is green. The sky is blue. The sun is yellow. Here we go. There and back again. " * 800
"\nWhat is the pass key?"
# The prompt is approximately 20k tokens long. You can try longer prompts by increasing the multiplie
)

chat_response = client.chat.completions.create(
    model="Qwen/Qwen2.5-7B-Instruct-1M",
    messages=[{"role": "user", "content": prompt}],
    temperature=0,
)
print("Chat response:", chat_response.choices[0].message.content)
```

其他选项

对于更高级的使用方式，可以探索如 [Qwen-Agent](#) 之类的框架。Qwen-Agent 使模型能够读取 PDF 文件或获得更多功能。

下一步

虽然 Qwen2.5-1M 系列为长序列处理任务带来了优秀的开源选择，我们也充分认识到长上下文模型仍有很大的提升空间。我们的目标是打造在长短任务中均能表现卓越的模型，确保它们在实际应用场景中真正发挥作用。为此，我们正深入研究更高效的训练方式、模型架构和推理方法，力求使这些模型即使在资源有限的环境中也能高效部署并且获得最佳的性能效果。我们坚信，这些努力将为长上下文模型开启全新的可能性，大幅拓展其应用范围。我们将持续突破这一领域的边界，敬请期待！